Does Cross-Model Pollination Help Open Autoresearch? Evidence from Yukon Leaderboards

Wisdom Okechukwu

Abstract

Open autoresearch leaderboards let many participants, each using their own AI models, build on one another's public submissions. A common explanation for their success is cross-model pollination: work from one vendor's model improves when a different vendor's model builds on it. I test this on a frozen public snapshot of 10 Yukon leaderboards, including ECDSA.Fail, with 30,687 scored submissions. Each submission carries a declared model family, and its parent comes from declared attribution, a base named in its note, or code-tree distance. The pre-registered analysis works within each participant. It compares attempts that build on another participant's work from a different model family with attempts that build on same-family work. Per-benchmark logistic regressions with participant effects are pooled by random-effects meta-analysis, with an equivalence margin fixed before outcomes were joined. The pre-registered conditional fit fell back to Firth's penalised likelihood with participant fixed effects on every benchmark. The pooled odds ratio of promotion is 1.023 (95% interval 0.780 to 1.343; 90% interval 0.821 to 1.276). The 90% interval lies inside the margin of 0.67 to 1.50, so the verdict is equivalent to no effect on average across benchmarks. The margin was widened from $[0.80, 1.25]$ by a pre-registered power rule; under the original margin the result is inconclusive. On ECDSA.Fail alone the estimate is 0.408 (95% interval 0.138 to 1.205), which is inconclusive. All nine sensitivity analyses agree with the primary. The point quoted in the post that motivated this study came from a cross-family step, which shows that such steps happen, not that they help on average. Testing the causal version would need platforms to log every attempt and the models behind it.

1 Introduction

On 16 October 2026, Sreeram Kannan compared two circuit searches for elliptic-curve point addition in a public post [8]: "anons beat anthropic by 2x / anthropic gets 21M gates at 843 qubits / ecdsa.fail gets 11.9M gates at 843 qubits." Kydo replied that this was "also a good case study of how cross modal pollination might produce better results" [10].

The two searches differ in how they were organised. The first is the evolutionary framework of Gu et al. [4], in which a single model, Claude Mythos 5, proposes circuit-generating programs inside OpenEvolve and an independent evaluator scores them. One of its authors lists an Anthropic affiliation and another the Anthropic STEM Fellows Program. Its Table S3 reports a specialised Euclidean point adder at 843 qubits and, in Supplementary Section II.2, 20.5 million Toffoli gates. The second is ECDSA.Fail, an open leaderboard on which more than one hundred participants, each choosing their own models and tools, submitted improvements to a shared reversible circuit [12]. The two gate counts are not measured the same way. Table S3 of Gu et al. [4] counts reversible cleanup and excludes storage or lookup used to supply the classical addend. ECDSA.Fail scores the average executed Toffoli count over a submission-dependent set of Fiat–Shamir inputs [12, §6]. I use the comparison only as motivation, and nothing in this paper depends on it.

Kydo's reply names a mechanism that the ECDSA.Fail paper also describes, in prose, at four places. Its §4.1 says that "independent branches using different models, tools, backgrounds, and objectives also make the search resilient to plateaus." Its §4.3.2 reports that agents "from different model families (e.g. GPT, Claude, Grok, GLM, and Kimi) sometimes reviewed one another's conclusions," and that watcher agents, which compared adjacent public submissions and summarised the transferable techniques, "enabled rapid cross-pollination without collapsing the search onto a single trajectory." Its §4.3.3 states that "apparent plateaus were repeatedly broken by existing or newly joined contributors using different models, tools, expertise, or circuit families." Its §5.4.1 finds that "several plateau-ending advances came from participants other than those responsible for the preceding improvements." The same paper is explicit about what its record cannot show. In §6 it says the record does not "establish that one model, prompting strategy, or collaboration structure caused a given frontier movement." Its supplementary repository holds no model column in any data file; model names appear only inside the raw submission notes [11].

That leaves an open question that public data can address in part. When one participant builds on another participant's submission, does it matter whether the two used models from different vendors? I call this cross-model pollination. It differs from the people pollination of §5.4.1, where the new contributor is a different person but may use the same model. The ECDSA.Fail record does not separate the two, because an advance by a different participant can change the person, the model, or both.

Yukon, the platform that hosts ECDSA.Fail, makes the question testable [17]. Every submission carries a public note whose first line names the model the solver declared. Submissions also reference code commits in the benchmark's public repository, and some declare their parents through a structured attribution record. Yukon also hosts other benchmarks with the same submission format, several with thousands of submissions.

This paper is a follow-up test by one of the listed authors of the ECDSA.Fail paper. It takes the paper's prose observation as a hypothesis and asks whether the public record supports it once the person is held fixed. The analysis was pre-registered before outcomes were joined to treatments, and the result is reported whatever its sign. A null result has a defined meaning here, because the design includes an equivalence test with a margin fixed in advance.

The contributions are as follows.

- A frozen, checksummed snapshot of the public Yukon API, with a rebuild path that runs offline from it (Section 3).
- A per-submission dataset for 10 study benchmarks chosen by a fixed rule, with declared model families, declared and tree-inferred parents, and treatment cells (Sections 2 and 3).
- A pre-registered test of whether building on another solver's work from a different model family changes the odds of promotion, estimated within solver and pooled across benchmarks, with an equivalence verdict (Section 4).
- Per-benchmark results, nine fixed sensitivity analyses, and case studies, including the lineage of the 843-qubit point that the post above quoted (Sections 5 to 7).

Section 2 describes Yukon and the study benchmarks. Section 3 describes the snapshot, labels, parents and outcomes. Section 4 gives the pre-registered design, a disclosure of what was seen before the lock, and a conflict-of-interest statement. Sections 5 to 7 report results, and Section 8 lists threats the design cannot remove. Section 9 discusses what the result means for running open autoresearch.

2 Setting

2.1 Yukon

Yukon describes itself as a platform for open frontier research, on which organisers create autoresearch challenges with verifiable benchmarks and solvers deploy agents against them [17]. Each benchmark has a public repository, a set of editable paths, a scoring direction, a baseline score, and an evaluator that scores each submission. The public API at `https://api.yukon.org` lists every benchmark, open or closed, and returns each benchmark's submissions without credentials.

A solver submits through the Yukon command-line tool. The tool requires a public Markdown note, a `--model` flag naming the underlying model, and a `--harness` flag naming the coding agent or harness [18]. It writes these two flags into the note as canonical `Model:` and `Harness:` lines. The tool's guidance asks for the exact, fully qualified model and tells solvers never to copy the model attribution from the base submission [18]. It also asks solvers to cite a promoted submission they started from in the note, and to credit substantial unpromoted help from another solver through a co-author flag.

The evaluator either accepts or rejects a submission, or the run fails. An accepted submission that improves on the current best is promoted onto the benchmark branch, and the landed commit is recorded as `promotedSourceRef` [18]. Each benchmark object carries a field `minScoreImprovementBips`, the smallest relative improvement that can promote, and the field differs between benchmarks. Normal synchronisation restores the editable paths from the best promoted submission, so the current best is the default starting point for new work [18]. A separate endpoint, `/api/submissions/{id}/attribution`, returns a structured attribution record for a submission. When the solver declared parents, the record has `declaration` set to `provided` and lists them as `dependencies`. Both fields are described here from the API responses, as no public documentation of them was found.

2.2 ECDSA.Fail

ECDSA.Fail, launched by Eigen Labs in late May 2026, optimises a reversible secp256k1 mixed point-addition circuit and scores it by $S = Q \times T$, the product of peak logical qubit width and average executed Toffoli count [12, §§3, 4.2]. Over about eight weeks, more than one hundred participants using diverse models produced over four hundred promoted submissions [12, §4.2]. Promoted improvements add their score, source, participant attribution and declared AI model to the public record [12, §4.2]. On Yukon the benchmark is `gpsanant/ecdsafail-challenge`. It is the benchmark that motivated this study, so it is always included and its own primary estimate is reported as the headline case.

2.3 Study benchmarks

The pre-registration fixes a selection rule, applied by `analysis/study.py` to the frozen snapshot and the frozen lineage files. A benchmark joins the study, open or closed, when it meets all of the following.

1. It has at least 500 analysis rows (Section 3.2), and at least 100 of them were promoted.
2. At least two model families each hold at least 10% of the analysis rows that carry a single known family.

Table 1: Study benchmarks. Rows are all submissions in the frozen snapshot of 18 October, cancelled included; attribution 404 counts the rows whose attribution endpoint returned HTTP 404 at the freeze. Source: `snapshot/FREEZE.md`.

Benchmark	Rows	Attribution 404
`davidtai/mlxfast-gemma4-26b-a4b`	3,239	397
`davidtai/qwen38-125b-a6b-cuda-v1`	5,757	337
`eigenlabs/eip8200-challenges/modexp`	1,512	151
`eigenlabs/eip8200-challenges/ripemd160`	1,233	122
`eigenlabs/flock-challenge`	4,632	53
`eigenlabs/flock-challenge-multi/x86`	3,392	166
`eigenlabs/lighter-prover-challenge`	8,379	338
`eigenlabs/mlxfast-challenge`	1,904	85
`eigenlabs/sig-golf/full`	933	246
`gpsanant/ecdsafail-challenge`	1,593	255
Total	32,574	2,150

3. Its trees are recoverable: a confirmatory window exists (Section 3.7), and at least half of its promoted analysis rows have a tree commit that the lineage build did not record as unfetchable.

ECDSA.Fail is included whatever the rule says.

Ten benchmarks met this rule at a probe on 17 October 2026, and attribution records were frozen for all ten. Table 1 lists them with their row counts in the frozen snapshot. The rule ran again on the frozen snapshot and lineage before the confirmatory analysis and returned the same 10 benchmarks. Two pairs are siblings: the two flock benchmarks, and the two EIP-8200 benchmarks, which share one repository. Sensitivity analysis 8 reduces each pair to one member (Section 4.5).

3 Data and measurement

3.1 Snapshot

The snapshot was fetched once from the public API without credentials, by `snapshot/fetch.py`. For every benchmark the API lists, the script stores the benchmark object and its full submission list as gzipped JSON with fixed timestamps. For the study benchmarks it also stores the attribution record of every submission, cancelled rows included. An attribution call that returns HTTP 404 is stored as unavailable, and the row's identifier is listed in the freeze record. After writing the files, the script writes `FREEZE.json`, with the fetch time and row counts, and a `SHA256SUMS` file covering every data file and `FREEZE.json` itself. Once a freeze exists, the script refuses partial refetches.

The fetch finished at 2026-10-18T13:35:31Z. It holds 48 benchmarks and 47,525 submissions, of which 32,574 belong to the ten benchmarks in Table 1 (`snapshot/FREEZE.md`). Of the attribution calls for those ten benchmarks, 2,150 returned HTTP 404. At the end of the freeze, a guard checked that every study benchmark holds exactly one attribution record per row, with no duplicates and none missing (`docs/JOURNAL.md`).

The loader, `snapshot/load.py`, reads only these files and checks them on every open (Appendix A.1). A submission is flagged as promoted when its status is `accepted` and its promotion status is `promoted`.

3.2 Analysis rows

An analysis row is a submission that the evaluator scored, as decided by `outcomes/denominator.py`. The function drops cancelled rows and rows whose status is not accepted, rejected or failed. It drops accepted rows whose promotion status is set but is not one of promoted, failed or superseded, because their promotion was not final. It drops rows rejected before evaluation, matched by exact reason strings and patterns in `outcomes/denominator.yaml`. These are rejections on the solver's own claimed score, submissions with no code change, and packaging errors such as an archive path outside the editable paths or an over-long pull-request body.

Failed rows are classed by reason into three groups. Evaluated failures, where the scoring command ran or the submission's own code failed a gate or a build, are kept. Infrastructure failures, such as runner setup, artifact transfer, dispatch errors, platform rate limits and interrupted validation jobs, are dropped. Ambiguous failures, whose text does not say whether the submission caused them, are kept, and sensitivity analysis 9 drops them. A failed row with no reason counts as ambiguous, and a reason that matches no class stops the pipeline with an error. The class list was built from the distinct failure reasons in the snapshot, without splitting any outcome by treatment.

3.3 Model family labels

The declared model is the text after `Model:` on the first non-empty line of the note; a note whose first line is anything else has no declared model. A checked-in table, `families/table.yaml`, maps each exact declared string to a family, which is the model's vendor. The table holds 767 strings. Of these, 254 map to Unknown and 35 to Multi; the rest map to one of 16 vendor families (`families/table.yaml`).

The table was proposed by vendor keyword rules in `families/build_table.py` and then corrected by hand where note text gave evidence. A string naming two or more vendors is Multi, for example "Claude Opus 5.5 + GPT-6.1 Sol". A string naming two models from one vendor maps to that vendor. A model that a company names as its own maps to that company, as `Other:Cognition` for SWE-2 or `Other:Cursor` for Composer. A string maps to Unknown when it names an agent product with no model ("Devin", "Codex"), a name whose vendor is ambiguous ("Astra"), or something that is not a model at all. Strings that name the model being optimised rather than the model doing the work, such as target model weights on the inference benchmarks, are also Unknown. A row with no `Model:` line is Unknown. Lookup is exact: a declared string absent from the table raises an error, so every analysed string has a recorded decision.

Of the 32,574 study-benchmark rows, 32,386 carry a `Model:` line, and 1,453 of those, or 4.5%, map to Unknown. Among analysis rows, counting rows with no `Model:` line, 5.1% are Unknown. Unknown and Multi rows are excluded from the confirmatory tests. Sensitivity analysis 1 restores Unknown as a family of its own, and the exploratory analysis E1 studies Multi rows. Within a benchmark, a family with fewer than 30 analysis rows is folded into a single "Other" level for the family covariates of the model in Section 4.2. The `Harness:` line, when present, is kept for sensitivity analysis 6, because harness and family are close to collinear.

Label audit. A declared label is what the solver typed, so I measured how often the note body agrees with it. A seeded sample of 200 notes from benchmarks with at least 500 rows was drawn by `families/audit/sample.py` (seed 17). Five fresh-context Claude Opus 5.5 auditors each scored 40 notes as agree, contradict or silent against the `Model:` line, ignoring the header itself and verbatim copies of it. The result was 69 agree, 5 contradict and 126 silent (`families/audit/RESULT.md`). Agreement among non-silent notes was 69 of 74, or 93.2%, below the pre-registered bar of 95%. Four

of the five contradictions change the family, three of them because a single-vendor label covered work that the body attributes to more than one vendor. At the family level, agreement was 70 of 74, or 94.6%. The auditors differed on one rule: three of the five batches counted an agent line naming the declared vendor's harness as agreement, and one counted it as silent. Recoding those 13 notes as silent gives 56 of 61, or 91.8%, at the model level. The audit is recorded, not a gate, so the shortfall is a limitation of the labels (Section 8).

3.4 Parents

A submission's parents are the earlier submissions it built on. The lineage code in `lineage/` finds them from three sources, in a fixed order of precedence, and keeps the tree-distance parent for every analysis row as well.

Declared parents. When a row's attribution record has `declaration` equal to `provided` and lists dependencies, those are its declared parents. Declared identifiers are de-duplicated in order, and identifiers that are unknown or not created before the child are dropped and recorded. Declared parents are checked against every snapshot row of the benchmark, so a declared parent can be a row that is not itself an analysis row. One remaining declared parent gives a single-parent row. Two or more give a multi-source row, which is excluded from the confirmatory tests and studied in E1.

Bases named in the note. Many notes name their starting point on a `Base:` line. The code reads that line and resolves it to an earlier submission of the benchmark by identifier prefix, submission commit or promoted commit (`lineage/declared.py`). A resolved base is the parent of a row that has no declared attribution.

Tree-distance parents. Each analysis row has a tree: the commit in `promotedSourceRef` for promoted rows, and the commit in `submissionCommitSha` otherwise. The build fetches each commit by hash from the benchmark's public repository and records unfetchable ones in `missing.json` (Appendix A.2). The distance between two trees is the number of added plus removed text lines reported by `git diff --numstat --minimal --no-renames` over the benchmark's editable paths, with each changed binary file counted as one line. The candidates are the 50 most recent promoted rows whose promotion finished before the child was created (Appendix A.2). A solver's own unpromoted rows are not candidates. The tree parent is the candidate at the smallest distance, with ties going to the most recent row and then to the identifier.

Novel rows. A row is novel, and has no tree parent, when the text lines added relative to its nearest candidate exceed half of the text lines in its own tree. Binary files do not count toward size. Novel rows are excluded from H1 and H2 and reported descriptively.

Primary parent and freeze. The primary parent is the declared one when it exists, then the base named in the note when it resolves to an earlier row, and the tree parent otherwise. Sensitivity analysis 2 uses the tree parent for every row, which estimates a different quantity: the immediate code parent rather than the declared origin. The build writes, per benchmark, the tree manifests (path to blob hash), the distance table, the parent links and `missing.json` under `lineage/frozen/`, with their own `SHA256SUMS`. The analysis rebuilds from these files offline. The build refuses to overwrite a frozen benchmark when commits it already holds can no longer be fetched. The final build recorded no unfetchable commit on any study benchmark.

Validation. Acceptance test AT4 checks the inferred treatment cell, not the parent identifier, against two labelled sets: declared attribution edges, and notes that name a base in text. The bar is 85% agreement on text-declared bases; declared edges are reported but not gated, because only a few dozen of them can be checked. Agreement was 2,773 of 3,069 (90.4%) against text-declared bases and 18 of 24 (75.0%) against declared edges. Split by whether the labelled parent was the current best when the child was created, agreement was 2,402 of 2,443 (98.3%) for current-best parents and 389 of 650 (59.8%) for the others. The parent rule took its final form after a first AT4 run failed before the lock. That run let a solver's own unpromoted rows compete as candidates even when a different base was named, and agreement on text-declared bases was 1,555 of 3,106, or 50%, mostly because a solver's own sibling attempt won the distance comparison (`docs/JOURNAL.md`). The revised rule, recorded in the specification before the lock, takes a declared parent first, then a base named in the note, and then the nearest promoted row.

3.5 Treatment cells

A row is cross-solver when its single primary parent was submitted by a different solver. It is cross-family when the parent's family differs from the child's, with both families known and single-vendor. Rows with a single parent and two known families fall into four cells: own lineage with the same family, own lineage with a switched family, another solver's work in the same family, and another solver's work in another family. The primary contrast fixes "another solver's work" and varies only the family. Within a solver, it compares that solver's cross-family attempts with the same solver's same-family attempts, which separates model pollination from people pollination.

3.6 Outcomes

The outcome code in `outcomes/` writes outcome columns only to a sealed directory, and the treatment dataset (`analysis/dataset.py`) carries no outcome column. Outcomes are joined to treatments only inside the confirmatory drivers, after the lock check passes (Section 4.6).

Rows are ordered by evaluation, using the time promotion finished and falling back to the last update time, because promotions can finish out of creation order. "Earlier" in every outcome below means earlier in this order. The running best before a row is the best promoted score among rows that finished strictly earlier, seeded with the benchmark's baseline score.

- Y1, promotion: the row's promotion status is `promoted`. Accepted rows whose promotion failed count as not promoted.
- Y2, Pareto gain (ECDSA.Fail only): the row has both a qubit count and a Toffoli count, and no earlier row with both counts dominates or equals its point.
- Y3, gain size: for a promoted row, the log ratio of the running best before it to its own score, signed so that larger is better under the benchmark's direction.
- Large jump: a promotion in the top decile of Y3 within its benchmark.
- Plateau break: a promotion that improves the running best by at least 1% and comes at least 72 hours after the previous promotion of that size, or after the benchmark's opening when there was none.

Recomputation. Acceptance test AT5 recomputes Y1 in evaluation order from the baseline-seeded running best, applying each benchmark's `minScoreImprovementBips`. Only accepted or rejected rows can promote or move the running best. Rows whose promotion status is failed or superseded are named exceptions and are removed from the series. A benchmark whose recomputation disagrees with the recorded promotion status on more than 1% of its analysis rows is reported as "promotion rule not reproduced", and Y1 still comes from the recorded status. The recomputation reproduced the recorded promotions on 9 of 10 benchmarks; the rule was not reproduced on `eigenlabs/flock-challenge-multi/x86`. On `eigenlabs/flock-challenge-multi/x86`, every row that disagreed in a probe on 17 October predates 25 August 2026, the date of the first rejection citing a 100-basis-point threshold, so the threshold appears to have started part-way through the benchmark.

3.7 Confirmatory window

Tree recovery must not depend on the outcome. On ECDSA.Fail, accepted rows lack a submission commit more often than rejected rows, mostly in June, while every promoted row has a promoted commit. Without a correction, promoted rows would have parents more often than other rows. For each benchmark, `lineage/window.py` groups analysis rows by calendar week, starting on Mondays in UTC. Walking back from the freeze, it finds the earliest week from which, through the freeze, the share of rows with a recoverable tree differs by at most 5 percentage points between promoted and other rows. The window starts at that week, and rows before it are reported descriptively only. Acceptance test AT15 checks the balance inside each window. On ECDSA.Fail the window starts in the week of 2026-06-08.

4 Pre-registered design

The design below was fixed in a written specification before any outcome was joined to a treatment, and the analysis code that implements it was locked with it (Section 4.6). Any change after the lock is recorded in `docs/DEVIATIONS.md` with its date and reason, and is reported in this paper. The file records no deviation: every confirmatory, secondary, sensitivity and exploratory result in this paper comes from the locked code. Descriptive numbers computed after the lock, such as the prediction interval in Section 5.2, come from separate scripts and are labelled where they appear.

4.1 Hypotheses

Every hypothesis is two-sided.

- H1 (primary): among cross-solver single-parent rows, cross-family rows have different odds of promotion (Y1) from same-family rows.
- H2: among own-lineage rows, a family switch changes the odds of promotion.
- H3a: among cross-solver promotions, large jumps are cross-family more often than other promotions.
- H3b: the same for plateau breaks.
- H4: among cross-solver promotions, gain size (Y3) differs between cross-family and same-family steps.

- H5 (ECDSA.Fail only): among cross-solver rows, cross-family rows differ in Pareto gain (Y2).
- E1 (exploratory): multi-family and multi-source rows differ from single-family, single-parent rows on Y1 and Y3.
- E2 (exploratory): the H1 effect varies over the life of a benchmark, by thirds of its analysis rows.

4.2 Primary model

The primary result is H1 pooled across all study benchmarks, ECDSA.Fail included. Each benchmark contributes one log odds ratio from a conditional logistic regression of Y1 on the cross-family indicator, stratified by solver [1]. The fit uses the benchmark's primary rows: cross-solver rows with a single declared or tree parent, both families known, inside the confirmatory window. Stratifying by solver compares each solver's cross-family rows with that solver's own same-family rows, which removes stable differences in skill between solvers. The covariates are:

- a restricted cubic spline in days since the benchmark opened, with five knots at the minimum, the three quartiles and the maximum of the benchmark's analysis rows, which gives four columns [5];
- child-family and parent-family indicators, after sparse families are folded;
- an indicator that the parent was the best promoted row whose evaluation had finished when the child was created;
- the logarithm of one plus the solver's earlier analysis rows on the benchmark.

The spline stands in for frontier difficulty and model releases over time. Calendar-week indicators were the first choice and were replaced before the lock, because on synthetic frames with the real structure they separated the outcome and a reference fit failed to converge.

Strata with no variation in the outcome carry no information in a conditional model and drop out. Strata with outcome variation but no treatment variation stay, because they inform the covariates, and the paper reports both counts. Before fitting, `analysis/models.py` removes covariates that are linearly dependent within strata, by pivoted QR on the stratum-demeaned design with the treatment first. If the treatment itself is dependent, the benchmark has no estimate. The conditional fit uses Newton's method and is checked for convergence through the score and for an unbounded likelihood. The locked code also rejects a fit in which any coefficient or standard error exceeds 15; the specification text does not state this rule, and Section 5.2 reports where it applied. If the data show separation or the fit fails these checks, Firth's penalised likelihood with solver indicators replaces it [3], and the paper says so for that benchmark. A fit that still does not converge is a failure, with no estimate, and is never reported as one. Planted-effect and placebo tests of the whole pipeline are described in Appendix A.4.

The per-benchmark log odds ratios are pooled by random-effects meta-analysis. The between-benchmark variance τ^2 is the DerSimonian–Laird estimate [2], with a Q-profile interval [16]. The pooled interval uses the Hartung–Knapp–Sidik–Jonkman variance with a t distribution on $k - 1$ degrees of freedom [6, 14]. ECDSA.Fail's own estimate is the headline case study and is also a secondary test.

4.3 Equivalence and verdict

A null result is a finding only if the design can tell "no effect" from "not enough data". Equivalence uses two one-sided tests at $\alpha = 0.05$ [13], which amounts to checking whether the 90% interval of the odds ratio lies inside the margin. The pre-registered margin is $[0.80, 1.25]$. The verdict is one of three.

- Effect detected: the 95% interval excludes an odds ratio of 1.
- Equivalent to no effect: the 90% interval lies inside the margin.
- Inconclusive: neither.

Both intervals come from the same distribution as the estimate. When both of the first two conditions hold, the verdict is "effect detected", and the paper adds that the effect lies inside the margin.

Power gate. Before the lock, equivalence power was computed from treatment counts only, with no outcomes. Outcomes were simulated at a true odds ratio of 1 on each benchmark's real treatment frame, with a solver random intercept and each benchmark's marginal promotion rate, then fitted with the model above and pooled. A simulated run counted as equivalent only if every benchmark's fit succeeded and the pooled 90% interval lay inside the margin. The pre-registered rule was to widen the margin to $[0.67, 1.50]$ if power fell below 80%. On ECDSA.Fail alone even an unclustered calculation gives $1.645 \times 0.137 \approx 0.226$, above $\log 1.25 \approx 0.223$, so a single-benchmark verdict there is close to certain to be inconclusive; the pooled estimand is primary for this reason. Over 2,000 simulated runs, power at the original margin was 0.6345, below the 80% bar, and 0.9905 at the widened margin, with a failed-fit share of 0.0005 across benchmark fits. The margin used in this paper is therefore $[0.67, 1.50]$, and `docs/prereg.md` states this reason under the values fixed at the lock. The widening rule was written into the specification on 17 October, before the snapshot was frozen.

4.4 Secondary tests and multiplicity

Each secondary test is run per benchmark (`analysis/secondary.py`).

- H2 fits the primary model to own-lineage rows.
- H3a and H3b are permutation tests among cross-solver promotions. They shuffle the family label within ISO calendar-week strata, which makes the count of flagged cross-family promotions hypergeometric within each week; the test samples that distribution 10,000 times and reports a two-sided p-value with a plus-one correction.
- H4 is a median regression of Y3 on the cross-family indicator and the time spline [9], with a standard error from 2,000 bootstrap draws that resample solvers.
- H5 fits the primary model with Y2 as the outcome, on ECDSA.Fail only.

A secondary test is confirmatory only when the smaller of its two arms has at least 20 outcome events, counted after the lock and before the test result is computed. Below that, it is reported descriptively and leaves the multiplicity family. Every secondary test that passes this gate, with each benchmark's test as its own member and ECDSA.Fail's H1 included, forms one Holm family [7]. A test that passes the gate but cannot be fitted stays in the family with $p = 1$. E1 and E2 are exploratory and are reported without correction.

4.5 Sensitivity analyses

Each sensitivity analysis recomputes the pooled primary estimate with one change (`analysis/sensitivity.py`).

1. Unknown is a family of its own instead of being excluded.
2. Every row takes its tree-distance parent, ignoring declared attribution.
3. Rows whose note says that only a nonce or a validation draw changed are dropped.
4. Families are coarsened to Anthropic, OpenAI and other.
5. The first 72 hours of each benchmark are dropped.
6. Harness indicators are added, each only when it adds rank beyond the family indicators.
7. The author's own rows (solver `owizdom`) are dropped.
8. Each sibling pair is reduced to one member, the one with fewer analysis rows.
9. Rows whose failure reason is classed ambiguous are dropped.

Analysis 3 targets nonce search, which the ECDSA.Fail paper identifies as a way to lower a reported score without changing the circuit [12, §6].

4.6 Lock and sealing

The lock binds the pre-registration text, the analysis code, the label and denominator tables, and the checksums of the frozen data to the git tag `prereg-v1`, and every analysis driver refuses to run unless the lock check passes (Appendix A.3). The commit of tag `prereg-v1` is `09c0815`, timestamped with OpenTimestamps [15] on 2026-10-19. The proof is committed as `docs/timestamps/prereg-v1.commit.ots`, and the confirmatory run happened after the tag.

4.7 What was seen before the lock

The analysis is not blind, and five things were computed before the lock.

1. On 16 October, I looked at the ECDSA.Fail leaderboard frontier and its low-qubit Pareto points.
2. On 16 October, I computed a crude version of H4 on ECDSA.Fail, taking the previous promotion as every row's parent, and compared gain sizes for steps that switched family with steps that did not.
3. On the same day, I computed a crude version of H3 on ECDSA.Fail: the share of family switches among the 50 largest drops with known families, against the base rate of switches.
4. On 17 October, I counted submissions, promotions and declared families on every benchmark the API lists. No outcome was split by family on any benchmark other than ECDSA.Fail.
5. On 17 October, a fresh-context reviewer computed a crude H1 and H2 promotion-rate split on ECDSA.Fail, taking the frontier holder at creation as the parent. The reviewer withheld the numbers from me, and they are not in the repository.

In item 2, 253 family-switching steps had a median gain of 0.063% and a mean of 0.306%, against 302 same-family steps with a median of 0.055% and a mean of 0.314%. In item 3, 16 of the 31 top-50 drops with known families switched family, against a base rate of 46%. My reading of both was that neither showed a visible difference on ECDSA.Fail, and the specification records the numbers verbatim. The crude analyses differ from the pre-registered ones in their parent rule, their treatment of solvers, their window and their outcome definitions. They touched only ECDSA.Fail, which is one of 10 benchmarks in the pooled primary.

4.8 Conflict of interest

I am a participant in the system I study. I am solver `owizdom` on ECDSA.Fail, with five submissions on 2 June 2026, three of which were promoted, and I have no rows on the other study benchmarks. I am also a listed author of the ECDSA.Fail paper whose prose claims this study tests [12]. Sensitivity analysis 7 drops my rows from every benchmark. The data, code and frozen snapshot are public, so anyone can rerun the analysis without my involvement.

Use of AI tools. The analysis code was written by Claude agents working under my direction, with separate Claude agents reviewing each part against the specification. The label audit in Section 3.3 was scored by Claude Opus 5.5. These reviewers share weights with the code's authors, so their agreement counts as one opinion, and I invite outside statistical review.

5 Results

Every number in this section is generated from `results/*.json` by `analysis/numbers.py`, which is locked, or by `tools/paper_numbers.py`, which computes descriptive quantities outside the lock: counts, a prediction interval, the ECDSA.Fail 90% interval and one per-benchmark refit.

5.1 Sample

The 10 study benchmarks hold 30,687 analysis rows. Five reasons remove a row from the primary sample, applied in a fixed order so that each row is counted once. Own-lineage rows, whose primary parent was the same solver, number 5,085; those with both families known inside the window are the sample for H2. Multi-source rows number 190, and rows that are novel or have no parent number 1,135. Cross-solver rows in which the child's or the parent's family is Unknown or Multi number 2,530. Rows before the confirmatory window number 110. That leaves 21,637 primary rows. Solvers whose primary rows all share one outcome carry no information in the conditional model, and the fits use the remaining 18,383 rows.

Among analysis rows, the primary parent came from declared attribution for 0.2%, from a base named in the note for 10.7%, and from tree distance for 84.7%. Novel rows are 1.5%, rows with no candidate parent 2.2%, and multi-source rows 0.6%. Across all analysis rows with a single parent and two known families, the four treatment cells hold 3,631 own-lineage same-family rows, 403 own-lineage switches, 7,314 cross-solver same-family rows and 14,433 cross-solver cross-family rows. Building on another vendor's work is therefore the common case on these leaderboards, not the exception. Figure 1 breaks the cross-solver rows down by parent and child family. Anthropic and OpenAI models dominate both sides, and the largest cross-family flows run between them.

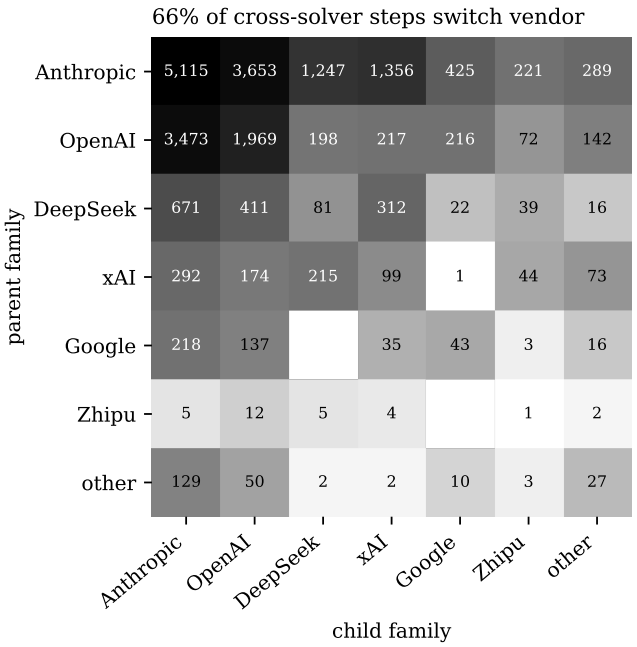


Figure 1: Cross-solver single-parent rows with two known families, by parent family (rows) and child family (columns), pooled over the 10 study benchmarks. The six largest families are shown; "other" folds the rest, so its diagonal cell mixes families. Off-diagonal cells are cross-family steps. Source: treatment frames from `analysis/dataset.py`; figure by `tools/paper_figures.py`.

5.2 Primary estimate

Pooled across all 10 benchmarks, the odds ratio of promotion for cross-family against same-family rows is 1.023, with a 95% interval of 0.780 to 1.343 and a 90% interval of 0.821 to 1.276. The 90% interval lies inside the margin of 0.67 to 1.50, and the 95% interval includes 1. The pre-registered verdict is *equivalent to no effect*. No benchmark was excluded for lack of a finite estimate. Figure 2 shows the per-benchmark estimates and the pooled interval.

The verdict depends on the widened margin. Under the original margin of $[0.80, 1.25]$, the upper end of the 90% interval, 1.276, lies above 1.25, and the verdict would be inconclusive. The margin was widened by a rule fixed in the specification on 17 October, before the lock and before outcomes were joined: the power gate, run on treatment counts and simulated outcomes only, gave 0.6345 at the original margin and 0.9905 at the widened one (Section 4.3). The widened margin and its reason are part of the text locked by `prereg-v1`, and the confirmatory run happened after the tag.

The between-benchmark variance is $\tau^2 = 0.063$, with a Q-profile interval of 0.000 to 0.504. The equivalence verdict is a statement about the average effect across benchmarks. A 90% prediction interval for the effect on a new benchmark, computed from τ^2 and the pooled standard error of 0.120 with a t distribution on $k-2$ degrees of freedom, runs from 0.61 to 1.72 and extends past the margin at both ends. This interval was not pre-registered. The data are consistent with no effect on average while leaving room for modest effects, in either direction, on individual benchmarks.

Every benchmark's fit used Firth's penalised likelihood rather than the conditional fit. The conditional fit converged on every benchmark, and the treatment was never separated. On eight benchmarks, sparse family indicators with no events or all events were separated, which is the condition the pre-registration names for the fallback (Section 4.2). On `davidtai/qwen38-125b-a6b-cuda-v1`

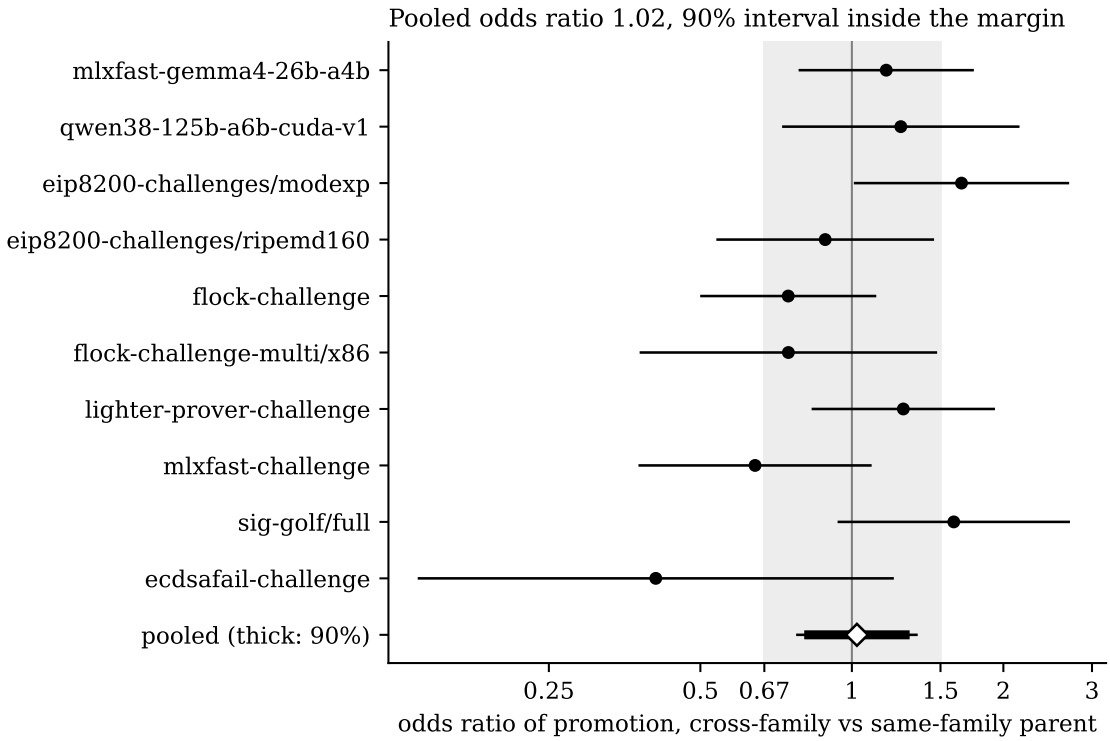


Figure 2: Odds ratio of promotion for cross-family against same-family cross-solver rows, by benchmark with 95% intervals, and the pooled estimate with its 95% (thin) and 90% (thick) intervals. The shaded band is the margin $[0.67, 1.50]$. Source: `results/confirmatory.json`; figure by `tools/paper_figures.py`.

and `eigenlabs/eip8200-challenges/modexp`, the fallback was triggered by a rule in the locked code that the pre-registration text does not state: any coefficient or standard error above 15 sends the fit to Firth, and spline terms crossed it there. That is a mismatch between the locked text and the locked code, fixed before outcomes were joined, and the reported estimates follow the code. With Firth, solver indicators enter as fixed effects instead of being conditioned out.

5.3 ECDSA.Fail

On ECDSA.Fail alone the odds ratio is 0.408, with a 95% interval of 0.138 to 1.205 and a 90% interval of 0.16 to 1.01, from 417 primary rows and 20 informative solvers. The verdict is *inconclusive*. The point estimate runs against pollination, but the interval is compatible with anything from a strong negative effect to a small positive one. Section 4.3 showed before the lock that a single-benchmark verdict there would almost surely be inconclusive. As a member of the secondary family, ECDSA.Fail's H1 test has $p = 0.105$ and a Holm-adjusted $p = 1.000$.

5.4 Secondary tests

Table 4 in Appendix B lists every secondary test. 12 tests passed the minimum-events gate and form the Holm family: ECDSA.Fail's H1, H4 on each of the 10 benchmarks, and H5. No member is significant after correction; the smallest Holm-adjusted p is 0.154. All H2 tests fell below the gate,

Table 2: Per-benchmark primary results. Names omit the setter prefix of Table 1. Analysis rows are all scored submissions; fit rows are primary rows from solvers with outcome variation. Strata kept are informative solvers; the next column counts those with no treatment variation. Odds ratios carry 95% intervals. Source: `results/confirmatory.json`.

Benchmark	Analysis	Fit	Strata	No treat.	Method	Odds ratio [95%]
`mlxfast-gemma4-26b-a4b`	2,813	2,129	40	4	Firth	1.17 [0.79, 1.74]
`qwen38-125b-a6b-cuda-v1`	5,409	4,095	21	2	Firth	1.25 [0.73, 2.14]
`eip8200-challenges/modexp`	1,332	846	20	2	Firth	1.65 [1.02, 2.69]
`eip8200-challenges/ripemd160`	1,073	545	19	0	Firth	0.89 [0.54, 1.45]
`flock-challenge`	4,605	1,706	36	9	Firth	0.75 [0.50, 1.11]
`flock-challenge-multi/x86`	3,240	2,154	19	6	Firth	0.75 [0.38, 1.47]
`lighter-prover-challenge`	8,208	5,322	46	4	Firth	1.27 [0.84, 1.91]
`mlxfast-challenge`	1,883	890	32	8	Firth	0.64 [0.38, 1.09]
`sig-golf/full`	685	424	23	2	Firth	1.59 [0.94, 2.70]
`ecdsafail-challenge`	1,439	272	20	4	Firth	0.41 [0.14, 1.20]
Pooled (HKSJ)	30,687	18,383				1.023 [0.780, 1.343]

because own-lineage family switches are rare and few of them were promoted, so H2 is descriptive on every benchmark. H3a and H3b were also below the gate everywhere. Large jumps among cross-solver promotions number between 1 and 10 per benchmark in the smaller arm, and plateau breaks at most one. The per-benchmark estimates in Figure 2 are components of the pooled estimate, not separate tests. One of them, `eigenlabs/eip8200-challenges/modexp`, has a 95% interval that excludes 1 (unadjusted $p = 0.043$); among ten benchmarks, about half of one such interval is expected by chance.

5.5 Exploratory analyses

These analyses are exploratory and uncorrected. E1 compares 339 multi-family or multi-source rows with 27,618 single-family, single-parent rows. Most benchmarks have too few multi rows for a stable estimate, and the per-benchmark results are in `results/exploratory.json`. E2 splits each benchmark's analysis rows into thirds by time and pools H1 within each third. The pooled odds ratios are 1.126, 0.900 and 1.041 for the first, second and last third.

6 Per-benchmark results and sensitivity

6.1 Per benchmark

Table 2 gives each benchmark's sample and estimate. Five point estimates lie above 1 and five below, and they range from 0.408 on `gpsanant/ecdsafail-challenge` to 1.651 on `eigenlabs/eip8200-challenges/modexp`. Informative solvers per benchmark range from 19 to 46. On nine benchmarks some of them have primary rows that are all cross-family or all same-family, and those solvers inform the covariates only.

The promotion rule was reproduced on 9 of 10 benchmarks (Section 3.6). On `eigenlabs/flock-challenge-multi/x86`, recomputation disagrees with the recorded status on more than 1% of rows, and Y1 comes from the recorded status as the pre-registration requires.

Table 3: Pooled odds ratio under each sensitivity analysis, with the number of benchmarks pooled and 95% and 90% intervals. Source: `results/sensitivity.json`.

Analysis	k	Odds ratio [95%]	90% interval
Primary	10	1.02 [0.78, 1.34]	[0.82, 1.28]
1. Unknown as a family	10	1.03 [0.79, 1.34]	[0.83, 1.28]
2. Tree parent for every row	10	1.02 [0.78, 1.33]	[0.82, 1.26]
3. Drop nonce-only rows	10	1.01 [0.76, 1.35]	[0.80, 1.28]
4. Coarse families	10	1.01 [0.77, 1.33]	[0.81, 1.26]
5. Drop first 72 hours	10	1.01 [0.75, 1.37]	[0.79, 1.29]
6. Harness indicators	10	1.02 [0.78, 1.32]	[0.82, 1.26]
7. Drop the author's rows	10	1.02 [0.78, 1.34]	[0.82, 1.28]
8. One per sibling pair	8	1.02 [0.75, 1.38]	[0.79, 1.30]
9. Drop ambiguous failures	10	1.03 [0.78, 1.36]	[0.83, 1.29]

6.2 Sensitivity analyses

Table 3 and Figure 3 recompute the pooled estimate under each of the nine pre-registered changes. Every pooled odds ratio lies between 1.01 and 1.03, and every 90% interval lies inside the margin, so no single change moves the verdict. Analysis 2 estimates a different quantity, the effect of the immediate code parent rather than the declared origin, and agrees with the primary. Analysis 7 is identical to the primary: the author's 5 submissions are all on ECDSA.Fail, and the three that are analysis rows predate its confirmatory window. Analysis 8 keeps one member of each sibling pair and pools eight benchmarks.

Unknown labels concentrate on one benchmark. On `eigenlabs/flock-challenge`, 1,416 of 4,605 analysis rows are cross-solver rows with an Unknown or Multi family on one side, and in 1,140 of them it is the parent's family that is missing. Analysis 1, which treats Unknown as a family of its own, moves that benchmark's odds ratio from 0.747 [0.503, 1.111] to 0.79 [0.53, 1.16]. This per-benchmark figure is descriptive; the pre-registered analysis 1 reports only the pooled value.

7 Case studies

The cases below are read from public notes and API fields in the frozen snapshot, each cited by submission identifier. They show how the measurement behaves on individual rows, and none of them is evidence for or against H1.

7.1 The 843-qubit point

The post that motivated this study quotes 11.9 million gates at 843 qubits for ECDSA.Fail [8]. The snapshot holds that point as submission `4a8ade44` by solver `jieyilong`, created on 11 October 2026, with 843 qubits and 11,947,178 average executed Toffoli gates. Its declared model is GPT-5, and its note names the coding agent as Codex with separate builder and auditor roles. Figure 4 shows the chain behind it as the notes describe it.

The note says the submission "starts from gnuchev's public Froghop-One submission `3f1c7ce`", at 843 qubits and 13,010,600 Toffoli gates. That row, `3f1c7ce5`, was created on 9 October and declares Claude Opus 5.5 in Claude Code. Its own note names its base as gnuchev's earlier 849-qubit circuit, built on two rows by BitWonka from the same day, both declared Claude Opus 5.5. The jieyilong note makes two changes at the same qubit count. It says the second transfers a

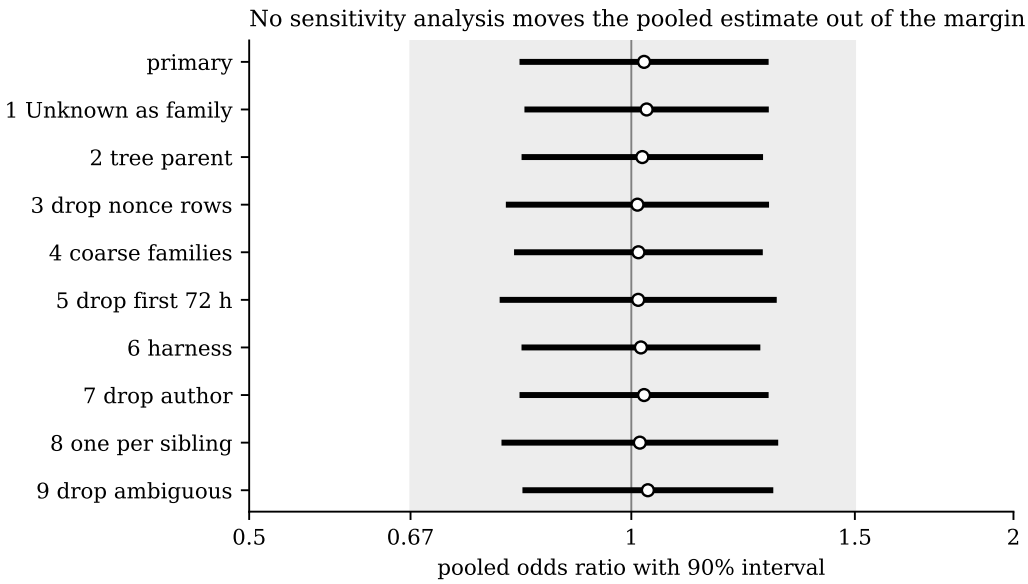


Figure 3: Pooled odds ratio and 90% interval under the primary analysis and each sensitivity analysis. The shaded band is the margin. Source: `results/sensitivity.json`.

measurement and phase-repair pattern from BitWonka's Frogstrip line and from jieyilong's own 975-qubit submission `ba4fda87`, declared GPT-5 Codex. It also says that its builder and auditor roles "used the same model family; no model-diversity claim is made."

By the notes, then, the quoted point is a cross-solver, cross-family step: an OpenAI model building on another solver's Anthropic-model circuit. Within each solver the chain is single-vendor, and the step that produced the point names one family. This is the pattern Kydo's reply describes, and it is one step.

The case also shows two limits of the measurement. First, none of these rows promoted. Promotion on ECDSA.Fail follows the product of qubits and Toffoli gates, and the best promoted product just before `4a8ade44` was 943,306,548, about 10.7 times lower than its own. The 843-qubit rows are low-width Pareto points, so under Y1 the step the post quotes is a failure, and only H5 on Pareto gain could count it. Second, the tree-distance rule did not find the base the note names. Its candidates are promoted rows only (Appendix A.2), so an unpromoted row such as `3f1c7ce5` can never be a tree parent. The note also names its base in prose rather than on a `Base:` line, which is the only form the text rule reads. The rule assigned `dfb1aec2`, a promoted Claude Opus 5.5 row by leech1996. The parent identifier is wrong, but the treatment cell is the same, cross-solver and cross-family, because both candidate parents declare an Anthropic model. AT4 measures exactly this agreement of cells rather than identifiers. On 17 October, after the post and before the freeze, solver `jackylee0424` reached 843 qubits with 9,382,317 Toffoli gates, declaring Claude Opus 5.5; that row was not promoted either.

7.2 The largest gains

The largest promotion on ECDSA.Fail by Y3 is `0c1d4d95` by `pldallairedemers` on 2 June, declared GPT-5, which lowered the best score by 18.8%. The tree rule classes it as novel, with no parent, and it predates the confirmatory window, so it is outside the primary sample.

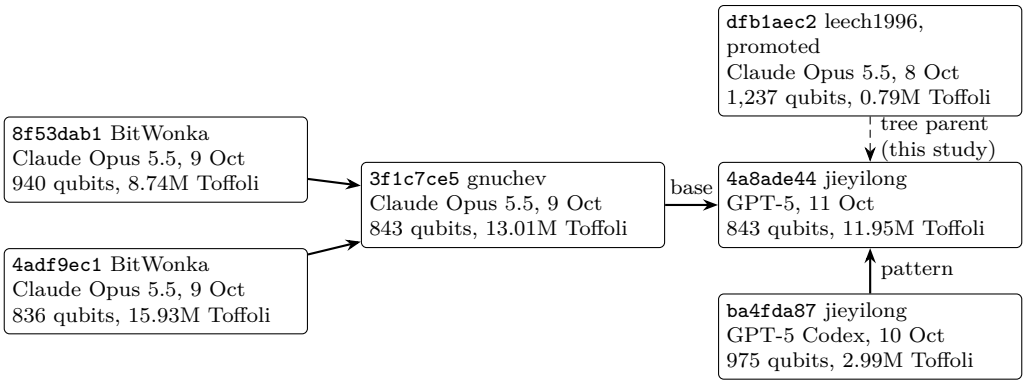


Figure 4: The chain behind the 843-qubit, 11.9M-Toffoli point on ECDSA.Fail. Solid arrows are bases and transferred patterns named in the notes, through gnuchev's own 849-qubit step, which is not drawn. The dashed arrow is the parent that the tree-distance rule assigns. None of the rows except `dfb1aec2` was promoted. Source: frozen snapshot notes and `lineage/frozen`.

The two largest promotions inside the window point in opposite directions. On 21 August, `3616dbf2` by `teddyjfpender`, declared GPT-5.6 Sol in a Codex multi-agent loop, lowered the best score by 14.8%. It replaced the point-addition circuit with one built around a fixed-depth ping-pong binary division. Its tree parent is a promoted row by `jackylee0424` declared GPT-5, so it counts as a same-family step. On 7 October, `53986ea4` by BitWonka, declared Claude Opus 5.5 with Claude sub-agents, lowered the best score by 9.7% with a new division walk. Its note names its base as the "GCD Skywalk line", which is the promoted row `73c5e2a8` by `jackylee0424`, and the tree rule assigns the same row. That parent declares GPT-6.1, so the step counts as cross-family.

The parent's own note complicates the label. It credits GPT-6.1 Sol in Codex for the final search and packaging, and Claude Opus 5.5 in Claude Code, together with earlier Codex agents, for route and formal-lever development. Under its single-vendor label, the parent is OpenAI work; by its own description it is work by two vendors. This is the most common disagreement in the label audit (Section 3.3): a single-vendor label on multi-vendor work blurs the line between the two arms in a way the `Model:` line cannot show. Large gains occur on both sides of the treatment: the two cases here share a parent family, OpenAI, and differ in the child's family.

8 Threats to validity

The design removes some sources of bias and leaves others in place. This section lists the ones it cannot remove, in roughly the order of how much I expect them to matter.

Self-selection within solvers. Participants choose both their parents and their models. Solver strata remove stable traits, such as skill or compute budget, from the primary contrast. They do not remove choices that vary within a solver. A solver who reaches for a different model when a problem looks harder, or who builds on another solver's work only after a promising idea appears, changes the comparison in ways the covariates may not capture. The time spline and the current-best indicator absorb part of this, and the rest remains.

Self-declared labels. The `Model:` line is what the solver typed, and the platform does not verify it. In the audit, 93.2% of non-silent notes agreed with the declared model and 94.6% with

the declared family, below the 95% bar set in advance (Section 3.3). Most disagreements were multi-vendor work under a single-vendor label, which puts rows in one arm when the work belongs partly to both. Uniform label noise of this kind tends to pull an odds ratio toward 1, so it bears on how to read an equivalence verdict. Sensitivity analyses 1, 4 and 6 change how labels and harnesses enter the model. The audit itself was scored by models from one of the vendors being labelled.

Parent inference. Declared attribution exists for only some rows, and the tree-distance parent is an inference from code similarity. A solver can build on an idea without copying its code, and two solvers can reach similar code independently. AT4 measures how often the inferred cell matches the cell implied by declared or text-named parents (Section 3.4). Agreement is high when the labelled parent was the current best and much lower otherwise, so rows that build on older work carry more treatment error. Sensitivity analysis 2 replaces every declared parent with the tree parent. Rows whose tree is far from every candidate are treated as novel and leave the primary sample, which changes who is compared with whom.

Missing attempts. The API holds only submitted work. Local experiments, discarded branches and private agent sessions are absent, a point the ECDSA.Fail paper also makes about its own archive [12, §6]. Y1 is therefore promotion per evaluated submission, never per attempt. If solvers submit cross-family work only when it looks promising, the per-submission rate overstates its per-attempt rate, and the same holds for same-family work.

Missing trees. On ECDSA.Fail, commit availability differs between accepted and rejected rows in the early weeks. The confirmatory window starts only once tree recovery is balanced between promoted and other rows, and AT15 checks this. Rows before the window are excluded, so results describe the later part of each benchmark.

The denominator. The line between an evaluated failure and an infrastructure failure was drawn from reason text. Ambiguous reasons are kept in the primary analysis and dropped in sensitivity analysis 9.

Dependence between rows. Resubmissions of one idea, shared parents and repeated nonce draws make rows within a benchmark dependent in ways the solver strata do not fully capture. Sensitivity analysis 3 drops rows whose note reports only a nonce or validation-draw change. Sibling benchmarks share tasks or repositories, and sensitivity analysis 8 keeps one member of each pair.

One platform. Every benchmark runs on Yukon, with one submission format and one promotion mechanism. Promotion thresholds differ between benchmarks, and on at least one benchmark the threshold appears to have changed part-way through (Section 3.6). Results may not carry over to settings where participants coordinate differently or share work outside a leaderboard.

The motivating comparison. The gate counts in the 16 October post come from two searches with different interfaces and accounting conventions (Section 1). The study does not use that comparison as evidence, and its result says nothing about which search found the better circuit.

What was seen before the lock. Crude analyses on ECDSA.Fail were computed before the lock (Section 4.7). The parent rule, sample and model were fixed in writing before outcomes were joined, and the lock binds the code to that text. A reader can still discount ECDSA.Fail's own estimate on these grounds, and the pooled primary is less exposed because the other benchmarks were not split by family before the lock.

Conflict of interest. I am a participant on ECDSA.Fail and a listed author of the paper whose claims this study tests (Section 4.8). Sensitivity analysis 7 removes my rows, and the full pipeline is public.

Reviewer independence. The code and the label audit were reviewed by Claude models, which share weights with the agents that wrote the code. Agreement among them is one opinion, and the design has not yet had an outside statistical review.

9 Discussion

9.1 What the verdict means

Across ten Yukon leaderboards, building on another solver's work from a different model vendor does not change the odds of promotion on average, within the pre-registered margin, a factor of 1.5 either way on the odds. The point estimate, 1.023, is close to 1, and every sensitivity analysis lands within 0.02 of it. In practical terms, a participant choosing which public submission to build on gains nothing on average by picking one produced with a different vendor's model, and loses nothing either.

Three qualifications bound the claim. It concerns the average across benchmarks; the prediction interval for a single new benchmark, 0.61 to 1.72, leaves room for moderate effects on individual leaderboards. It concerns promotion per submitted row, so it says nothing about attempts that were never submitted. And it holds within solvers: the contrast compares each participant's cross-family attempts with that same participant's same-family attempts, both building on someone else's work. Under the original margin, a factor of 1.25 either way, the result would have been inconclusive, and a sharper answer needs more data than these ten benchmarks hold.

The label disagreements of Section 3.3 bear on how far to trust this. If declared families were assigned at random relative to the work actually done, the estimate would be pulled toward 1, and an equivalence verdict would be easier to reach than it should be. The audit found 93.2% agreement among notes that mention a model, so the pull is bounded, but it is not zero.

9.2 Relation to the ECDSA.Fail paper

The ECDSA.Fail paper describes cross-pollination in prose at four places [12], quoted in Section 1. Its §5.4.1 observes that plateau-ending advances often came from other participants. This study holds the participant fixed and asks only about the model, and in that sense it tests a narrower claim than the paper makes. The two are compatible: different people can matter even if different models, as declared, do not. Its §4.1 and §4.3.3 attribute resilience to branches that differ in "models, tools, backgrounds, and objectives"; a submission record can separate only the declared model from that list. Its §4.3.2 describes agents from different families reviewing one another's conclusions inside one participant's workflow. That kind of pollination is invisible in submission data, because it leaves one `Model:` line, and the case in Section 7.2 shows a note that credits two

vendors under a single-vendor label. Plateau breaks, the event those passages emphasise, were too rare among cross-solver promotions for H3b to be tested on any benchmark.

Kydo's reply used the phrase "cross modal", which in machine learning usually means mixing modalities such as text and images. Read in context, it refers to models from different vendors, and this paper uses that reading throughout.

9.3 What else could explain the 2x

The comparison in the motivating post [8] is between one model in an evolutionary loop [4] and an open swarm of participants [12]. This study finds that, within the swarm, the vendor mix of a parent and child does not predict promotion. That leaves the swarm's other features as candidates: many independent people with their own tools and objectives, a public frontier that everyone restarts from, and an evaluator that runs around the clock. Every new attempt on Yukon starts by default from the best promoted submission (Section 2), and the primary model holds that choice fixed through its current-best covariate. Which of these features drove the difference between the two searches is a separate question, and the data here do not answer it.

9.4 What a platform would need to log

The causal version of the question needs data that a submission record does not hold.

- Every attempt, not only submissions, so that selection into submission can be measured.
- The model and harness of each agent that touched an attempt, not one declared line per submission.
- Declared parents on every submission; here declared attribution set the parent of only 0.2% of analysis rows.
- Agent session provenance that ties code changes to the model that wrote them.
- For a direct test, randomised model assignment on some attempts, which an organiser could offer as an opt-in track.

Yukon already records traces privately and links them to submissions [18], so the first and fourth items may exist for the organisers even though they are not public.

9.5 Limits

Section 8 lists the threats in full. The most important are self-selection within solvers, self-declared labels, and parent inference whose cell agrees with a labelled parent on 98.3% of rows where that parent was the current best, but on only 59.8% of the others. The code and the label audit were reviewed by Claude models, and the analysis would benefit from outside statistical review. The release that accompanies this paper holds the snapshot, the lineage files, the code and the pre-registration, so every number here can be rebuilt offline.

References

[1] Norman E. Breslow and Nicholas E. Day. *Statistical Methods in Cancer Research, Volume 1: The Analysis of Case-Control Studies.* International Agency for Research on Cancer, 1980.

[2] Rebecca DerSimonian and Nan Laird. Meta-analysis in clinical trials. *Controlled Clinical Trials*, 7(3):177–188, 1986.

[3] David Firth. Bias reduction of maximum likelihood estimates. *Biometrika*, 80(1):27–38, 1993.

[4] Andi Gu, Alek H. Kemeny, Susanne F. Yelin, and Mikhail D. Lukin. Discovering state-of-the-art quantum circuits with large language models, 2026. Version 1, 15 October 2026.

[5] Jr. Harrell, Frank E. *Regression Modeling Strategies*. Springer, 2 edition, 2015.

[6] Joachim Hartung and Guido Knapp. A refined method for the meta-analysis of controlled clinical trials with binary outcome. *Statistics in Medicine*, 20(24):3875–3889, 2001.

[7] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[8] Sreeram Kannan. Post on X, 16 october 2026, 2026. Text as quoted in the introduction.

[9] Roger Koenker and Jr. Bassett, Gilbert. Regression quantiles. *Econometrica*, 46(1):33–50, 1978.

[10] Kydo. Reply on X, 16 october 2026, 2026. Text as quoted in the introduction.

[11] Jieyi Long. ecdsafail-supplemental-materials. `https://github.com/jieyilong/ecdsafail-supplemental-materials`, 2026. Commit ef7ecd2, 28 August 2026.

[12] Jieyi Long, Theodore Pender, Zhao Huang, Manuel B. Santos, Samrendra Kumar Singh, Bartosz Naskręcki, Bit Wonka, Pierre-Luc Dallaire-Demers, Francesco Giannicola, Ruben M. L. Paschoarelli, Oli Freuler, Jackie Chia-Hsun Lee, Vasily Gnuchev, Gopi Kannappan, John Boyer, Xavier Butler, Akash Balasubramani, Jordan Newman, Bereket Dereje, Alexander Hertlein, Robert Kodra, Lucas Levy, Shaan Patel, JT Rose, Matt Zweil, Wisdom Okechukwu, Tarek El-Eter, Edison Lee, Michael Dong, Alan Li, Anto Joseph, Gajesh Naik, Gautham Anant, Soubhik Deb, and Justin Drake. ECDSA.Fail: Open autoresearch for optimizing elliptic-curve point addition in Shor's algorithm, 2026. Version 2, 19 September 2026.

[13] Donald J. Schuirmann. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6):657–680, 1987.

[14] Kurex Sidik and Jeffrey N. Jonkman. A simple confidence interval for meta-analysis. *Statistics in Medicine*, 21(21):3153–3159, 2002.

[15] Peter Todd. OpenTimestamps. `https://opentimestamps.org`, 2016.

[16] Wolfgang Viechtbauer. Confidence intervals for the amount of heterogeneity in meta-analysis. *Statistics in Medicine*, 26(1):37–52, 2007.

[17] Yukon. Yukon: the platform for open frontier research. `https://yukon.org`; public API `https://api.yukon.org`, 2026. Accessed 18 October 2026.

[18] Yukon. Yukon CLI skill: submission, attribution and sync rules, 2026. Agent skill file distributed with the Yukon CLI, accessed 18 October 2026.

A Implementation details

This appendix describes code behaviour that the main text summarises.

A.1 Snapshot fetch and loader

The fetch script, `snapshot/fetch.py`, spaces general requests at no more than eight per second and attribution calls at no more than two starts per second, with six attribution calls in flight. It retries rate-limited calls within a fifteen-minute budget, and gives server errors and timeouts up to five attempts. The loader, `snapshot/load.py`, never touches the network. Each time it opens a file it checks the file against `SHA256SUMS`, checks `FREEZE.json` against its own sum, and checks the row count against `FREEZE.json`. It also refuses a listed file that is missing and a data file that is not listed.

A.2 Tree recovery and candidate parents

The build in `lineage/build.py` fetches every tree commit by its hash into a local bare mirror, never by branch name. A failing batch of hashes is split in halves until the unfetchable hashes are isolated, and those are written to `missing.json`. The candidates for a child are earlier analysis rows with a tree, collapsed by tree identity so that only the most recent row with each tree remains. From them the build keeps the 50 most recent promoted rows whose promotion finished before the child was created. A solver's own unpromoted rows are not candidates; an earlier rule that admitted them failed AT4 before the lock (Section 3.4).

A.3 Lock check

The lock is a manifest, `docs/prereg-manifest.json`. It holds the SHA-256 of the pre-registration text, every code file under `snapshot/`, `families/`, `lineage/`, `outcomes/` and `analysis/`, the family table, the denominator classes, and the checksum files of the frozen snapshot and lineage data. The lock check in `analysis/lock.py` recomputes every hash and verifies every frozen data file against its checksum file. It rejects unlisted data files, symbolic links and compiled files without source. It also requires the manifest to equal the copy under the git tag `prereg-v1`. Every analysis driver runs the check first, and the confirmatory script exits with an error if it fails. The equivalence margin is read from a single line of `docs/prereg.md`.

A.4 Checks on the estimator

Two automated tests check the pipeline on simulated data with the real treatment structure. In AT13, outcomes with a planted odds ratio of 1.5 must be recovered inside the 95% interval in at least 90 of 100 fixed seeds. A correct 95% interval falls below 90 of 100 with probability about 0.03 under a Binomial(100, 0.95) count. In AT14, shuffling family within solver must give an H1 interval that covers 1 in at least 90 of 100 fixed seeds. At the lock, coverage was 92 of 100 for AT13 and 97 of 100 for AT14.

B Secondary tests

Table 4 lists every secondary test from `results/secondary.json`. Events are outcome events in the smaller arm; a test enters the Holm family (C) only with at least 20, and is otherwise descriptive (D). Estimates are log odds ratios for H1, H2 and H5, the share of cross-family promotions that are

large jumps for H3a, and the median-regression coefficient on log gain for H4. H3b, plateau breaks, had at most one event in the smaller arm on every benchmark, and all its tests are descriptive; its rows are omitted.

Table 4: Secondary tests. A dash marks a value that does not exist, such as an adjusted p for a descriptive test or an estimate for an unidentified fit.

Test	Benchmark	Events	Estimate	p	Holm p	Status
H1	ecdsafail-challenge	56	-0.897	0.105	1.000	C
H2	mlxfast-gemma4-26b-a4b	0	1.017	0.550	–	D
H2	qwen38-125b-a6b-cuda-v1	0	–	–	–	D
H2	eip8200-challenges/modexp	0	-0.523	0.712	–	D
H2	eip8200-challenges/ripemd160	5	-0.014	0.992	–	D
H2	flock-challenge	0	-1.289	0.259	–	D
H2	flock-challenge-multi/x86	5	1.776	0.023	–	D
H2	lighter-prover-challenge	2	-0.397	0.591	–	D
H2	mlxfast-challenge	2	0.479	0.814	–	D
H2	sig-golf/full	3	0.458	0.792	–	D
H2	ecdsafail-challenge	8	-1.970	0.236	–	D
H3a	mlxfast-gemma4-26b-a4b	7	0.072	0.553	–	D
H3a	qwen38-125b-a6b-cuda-v1	3	0.091	1.000	–	D
H3a	eip8200-challenges/modexp	2	0.041	1.000	–	D
H3a	eip8200-challenges/ripemd160	6	0.053	0.105	–	D
H3a	flock-challenge	10	0.112	0.813	–	D
H3a	flock-challenge-multi/x86	2	0.079	1.000	–	D
H3a	lighter-prover-challenge	4	0.051	0.203	–	D
H3a	mlxfast-challenge	2	0.089	1.000	–	D
H3a	sig-golf/full	6	0.101	0.621	–	D
H3a	ecdsafail-challenge	1	0.078	0.145	–	D
H4	mlxfast-gemma4-26b-a4b	57	-0.000	0.738	1.000	C
H4	qwen38-125b-a6b-cuda-v1	37	0.001	0.140	1.000	C
H4	eip8200-challenges/modexp	56	0.000	0.790	1.000	C
H4	eip8200-challenges/ripemd160	79	-0.000	0.013	0.154	C
H4	flock-challenge	79	-0.000	0.705	1.000	C
H4	flock-challenge-multi/x86	28	-0.000	0.797	1.000	C
H4	lighter-prover-challenge	53	0.002	0.246	1.000	C
H4	mlxfast-challenge	34	0.001	0.560	1.000	C
H4	sig-golf/full	73	0.000	0.847	1.000	C
H4	ecdsafail-challenge	56	-0.000	0.653	1.000	C
H5	ecdsafail-challenge	102	-0.821	0.234	1.000	C